

TaxTrace – Architecture

Pipeline

SAP ECC (AU)	Dynamics (UK)	Oracle (US)	NetSuite (NZ)
+-----+	+-----+	+-----+	+-----+
	[1. INGEST]	schema validation, reject on shape	
	v		
	data/raw	immutable, never edited in place	
	[2. PROFILE]	nulls, uniques, ranges, referential	
	v	integrity, categorical drift	
	reports/profile		
	[3. STAGE]	one file per source, typed, untouched	
	v	values, source column names kept	
	staging tables		
	[4. MAP]	source schema → canonical schema	
	v		
	[5. CLEAN]	trim, case fold, country and currency	
	v	standardisation, date parsing	
	[6. CANONICAL]	one transaction model, typed, keyed	
	v		
	[7. DATA QUALITY]	DQ001..DQ012, severity, thresholds	
	v	fail the run on CRITICAL	
	[8. RULES ENGINE]	decision table, precedence,	
	v	effective dating → tax_code + rate	
	[9. CALCULATE]	inclusive/exclusive, rounding,	
	v	currency → calculated tax	
	[10. VALIDATE]	golden dataset comparison	
	v		
	[11. RECONCILE]	source tax vs calculated tax,	
	v	variance classification	
	[12. ANOMALY]	deterministic + statistical	
	v		
	[13. EXCEPTIONS]	categorised, owned, ranked by value	
	v		
	[14. REPORT]	reconciliation summary, DQ scorecard,	
		exception register, run log	

Every stage is a pure function of its input plus configuration. That is what makes the pipeline **idempotent**: run it twice, get the same answer.

Layer contract

Layer	Owns	Must not
raw	Bytes as delivered	Be edited, ever
staging	Types, one row per source row	Change business meaning
canonical	One schema, standardised values	Apply tax logic
rules	Which rule applies	Do arithmetic

Layer	Owns	Must not
calculated reconciled	The arithmetic Comparison and classification	Decide which rule applied Change either input

INTERVIEW ALERT. Being able to say *why* those boundaries exist is worth more than being able to write any single stage. The reason is blast radius: when a number is wrong you need to know which layer to look in.

Traceability model

Every calculated row carries:

```
transaction_id, rule_id, rule_version, tax_code, tax_rate,
taxable_amount, calculated_tax, calculation_timestamp, engine_version, status
```

That tuple is the audit answer. Given a filed figure, you can name the rule, the version of that rule, and the source row.

Where cloud technology would sit in a real build

The local implementation is deliberately free. This is how it would map at a real client, and it is worth being able to describe in an interview.

Local here	Enterprise equivalent	What it would add
CSV in data/raw	Azure Data Lake Storage Gen2	Durable, versioned, access controlled landing zone
Python scripts	Azure Data Factory or Airflow	Scheduling, retries, dependencies, alerting, backfill
DuckDB	Azure SQL, Synapse or Snowflake	Concurrency, security, scale beyond one machine
Local pytest	Azure DevOps or GitHub Actions	Gate every merge on the regression suite
Local run log	Application Insights or Datadog	Alerting when exception rate moves
YAML in git	Same, plus an approval workflow	Segregation of duties on rule changes
Manual run	Databricks job or Function App	Elastic compute for month end peaks

Be honest in the interview. Say "I built this locally with DuckDB, and here is exactly how it would map onto Azure Data Factory and Synapse." That is a much stronger answer than pretending to Azure experience you do not have.

ETL versus ELT, in this project's terms

ETL transforms before loading. You would clean and apply tax rules in Python, then load only the finished result into the warehouse.

ELT loads raw first, then transforms inside the warehouse. TaxTrace is closer to ELT: raw lands untouched, staging and canonical models are built with SQL in DuckDB, and the transformations

are versioned in git.

ELT wins here for one specific reason that matters to tax: **you can always re-derive the answer from raw.** When a rate is discovered to be wrong six months later, ELT lets you reprocess history. ETL that discarded the raw input cannot.